scienceStack

Rigorous Scientific Engineering Workflows

Operational Manual for the Google Antigravity Plugin

Plugin Specification & Operational Guide

Version 0.23.0

Components:
12 Skills
6 Playbooks, 6 Principles
2 Subagents
4 Lifecycle Hooks
1 Directive Rule File

Domains:
PDEs, CFD, orbital mechanics
Stiff ODE kinetics
Numerical linear algebra

Adapted from the `pstack` architecture by Lauren Tan (poteto) at Cursor.
Specialized for equation provenance, dimensional analysis, and numerical verification in Google Antigravity.

Contents

Executive Summary **ii**

1 Installation and How to Use **1**
1.1 Installing the Plugin . . . 1
1.2 Skills . . . 1
1.3 Runtime Directives . . . 2

2 Architecture **3**
2.1 Subagents and Models . . . 3
2.2 Where Things Live . . . 3

3 The Six Principles **4**

4 Playbooks **6**

5 Safety Hooks and Tools **7**
5.1 PreToolUse Safety Gate . . . 7
5.2 Norms, order on your own problem, and what a tolerance means . . . 7
5.3 Building both sides of a comparison . . . 8
5.4 Verification hooks . . . 8
5.5 AST Auditor . . . 8
5.6 MMS and Jacobian Generators . . . 9
5.7 Reference Benchmarks . . . 9

6 Testing **11**

Recommended Literature & References **12**

Executive Summary

Generated code that satisfies local tests can still be wrong in the ways that matter to computational science: a flipped sign in a diffusion operator, an off-by-one in a stencil, or an explicit matrix inverse that squares the condition number. **scienceStack** packages a small set of enforceable rules, workflows, and tools for Google Antigravity that target exactly these failures. It is organized around six principles:

1. **Equation provenance**: every implemented formula carries its continuous LaTeX form, a literature citation, units, and sign conventions.
2. **Dimensional discipline**: both sides of every equation have the same dimensions; arguments of transcendental functions are dimensionless.
3. **Numerical stability first**: no explicit inversion, guarded denominators, and `log1p`/`expm1` for small arguments.
4. **Conservation laws**: Hamiltonian systems use symplectic integrators; energy, momentum, and angular momentum are tracked in tests.
5. **Physics-modular architecture**: pure physical laws live in stateless modules, decoupled from time steppers, meshes, and I/O.
6. **Analytical limits and manufactured solutions**: solvers are verified against closed-form solutions or the Method of Manufactured Solutions with explicit error norms.

The key of the plugin is strict verification and validation: a script, not an opinion, decides whether a solver matches a stated reference at a stated tolerance. Canonical problems, measured order of convergence, golden-data comparison, an evidence log bound to file hashes, ratchets that refuse a weakened test, and hooks that hold an agent to all of these.

Chapter 1

Installation and How to Use

1.1 Installing the Plugin

The plugin complies with Agent Plugins 1.0.0; its root manifest is `plugin.json`.

```
# 1. Validate compliance with Agent Plugins 1.0.0 (Strict mode)
agy plugin validate /home/seo3919/AI_skills/scienceStack_antigravity
   --strict

# 2. Install globally in Antigravity
agy plugin install /home/seo3919/AI_skills/scienceStack_antigravity

# Or workspace-local:
# Place scienceStack_antigravity in .agents/plugins/sciencestack
```

Verify installation status:

```
agy plugin list
# - sciencestack (enabled, 0.23.0)
#   Components: 12 skills, 2 agents, 1 rule, 4 hooks
```

1.2 Skills

Skills can be triggered via interactive slash commands or dynamically by Antigravity agents:

- `/science-mode <task>`: master workflow. Selects exactly one of six playbooks and enforces the six principles. User-invoked.
- `/setup-sciencestack`: writes per-role model choices to configuration. User-invoked.
- `astronomy-computing`: Kepler solvers, symplectic N-body dynamics, SO(3) coordinate frames.
- `accelerated-scientific-computing`: memory layout, SIMD vectorization and precision guards.
- `check-against-reference`: runs the user's solver on Kepler's equation, Lane-Emden, the Sod shock tube, Robertson kinetics, a Timoshenko cantilever, Hilbert linear systems or the Ghia lid-driven cavity, measures order of convergence (including 2-D Poisson and, with `convergence-user`, the user's own exact or manufactured solution), or compares its output with golden data, through `scripts/check_against_reference.py`. The script decides (exit status 0 pass, 1 fail, 2 nothing checked, 3 verdict withheld or manifest weakened); `convergence-ode` and `convergence-bvp` measure the observed order of convergence (errors at $N, 2N, 4N, \ldots$, the two finest resolved pairwise orders must reach the

claimed `-order` minus a margin, roundoff-level errors are discarded and too few resolved levels is a failure) against problems with exact solutions; `-repeat` defaults to 2 and fails if repeated calls differ, including the sign of zero, unless `-repeat-tol` allows it; every run is appended to a hash-chained evidence log bound to the SHA-256 of the solver; a tolerance loosened after a failure on unchanged files is withheld, and a justification sentence does not release it (release needs `tolerance-approvals.json`); `run-manifest` re-runs a committed `reference-checks.json` and, against a git baseline, refuses removed checks and raised tolerances without an approved reason; benchmarks whose error depends on the solver's discretisation have no default tolerance, so `-tol` is required, and a tolerance finer than the reference can resolve is refused.

- `run-tests`: runs unittest or pytest through `scripts/run_tests.py`, takes the counts from the framework (not from console text), fails when nothing ran or everything was skipped, and records the result bound to a SHA-256 of every tracked and untracked-not-ignored file; `run_tests.py fresh` exits 0 only if the latest run passed on exactly the current files.
- `test-ratchet`: `scripts/test_ratchet.py -baseline-ref REF` parses both versions of every test file and exits 3 on an unapproved looser tolerance (including a library default taking over), removed test, fewer assertions, new skip or xfail, or fewer parametrize cases; approvals with a reason go in `test-ratchet-approvals.json`.
- `numerical-stability-audit` also runs `audit_numerical_ast.py`, which warns about draws from an unseeded global random state, generators created without a seed, clock-, process-id- or entropy-derived seeds, and unseedable sources.
- `numerical-stability-audit`, `precision-audit`, `scientific-equation-verifier`, `golden-dataset-b` `benchmark-checklist`: audit and verification skills loaded on demand; each can also be run explicitly.

1.3 Runtime Directives

Directives are defined in `rules/AGENTS.md`: the core directives (provenance, stability, symplectic integration, modularity, reference checks, test ratchet, evidence logs) and absolute script locations.

Chapter 2

Architecture

2.1 Subagents and Models

Agent	Model	Tools	Role
`science-master`	`flash` / `inherit`	all	kernels, MMS tests, refactors, benchmark runners
`theory-reviewer`	`pro`	read-only	line-by-line audit of equations, signs, index order, invariants

Table 2.1: Subagents. `theory-reviewer` operates in a read-only context.

2.2 Where Things Live

```
plugin.json                          Agent Plugins 1.0.0 manifest
hooks/hooks.json                     PreToolUse, PostToolUse, and StopGate
    hooks
rules/AGENTS.md                      Antigravity runtime directives
agents/                              science-master, theory-reviewer
references/                          compact delegate principles (read by
    science-master)
skills/science-mode/                 playbooks/ (6) and principles/ (6)
skills/                              check-against-reference, run-tests,
    test-ratchet, ...
scripts/                             hooks, AST auditor, checker, evidence
    log, ratchet,
                                     MMS and Jacobian generators
benchmarks/                          reference solvers, Sod tolerance
    calibration
tests/                               unit and integration tests
```

Chapter 3

The Six Principles

Scientific Principle: Equation Provenance

Document the governing equation in the docstring, for example

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u \quad \text{[SI]},$$

with its literature source, coordinate system, and sign convention. The AST auditor reports scientific functions whose docstring has no LaTeX equation.

Scientific Principle: Dimensional Discipline

For every equation [LHS] = [RHS]. Arguments of exp, ln, sin must be dimensionless. Non-dimensionalize coupled systems with characteristic scales.

Scientific Principle: Numerical Stability First

For a linear system $A\mathbf{x} = \mathbf{b}$ the relative error obeys

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \lesssim \kappa(A) \left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|} \right).$$

Use `solve(A, b)` or a factorization rather than forming A^{-1}. Replace $\ln(1+x)$ by `log1p`, $e^x - 1$ by `expm1`, $1 - \cos x$ by $2\sin^2(x/2)$.

Scientific Principle: Conservation Laws

Hamiltonian systems $H(\mathbf{q}, \mathbf{p})$ use symplectic integrators (kick-drift-kick leapfrog, Yoshida splitting), which conserve a nearby shadow Hamiltonian $\tilde{H} = H + O(\tau^2)$ and show no secular energy drift. Track $E(t)$, $\mathbf{P}(t)$, $\mathbf{L}(t)$ in tests with an explicit tolerance. Explicit Runge–Kutta is not used for long-term orbit integration.

Scientific Principle: Physics-Modular Architecture

Forces, equations of state, and rate equations are pure, stateless functions in a `physics/` module that other solvers can import without a time stepper, mesh, or I/O.

Scientific Principle: Analytical Limits and MMS

For a differential operator $\mathcal{L}$, choose a smooth u_{mms}, compute $f = \mathcal{L}[u_{\text{mms}}]$ symbolically, solve $\mathcal{L}[u] = f$ numerically and check that the error norm falls with the expected order p under grid refinement:

$$p_{\text{obs}} = \frac{\ln(e_h / e_{h/2})}{\ln 2}.$$

Chapter 4

Playbooks

`/science-mode` selects and executes the specific playbook matching the task.

Playbook	Purpose
`equation-implementation`	Turn a published equation into verified code with LaTeX docstrings, dimension checks, and limit tests.
`numerical-verification`	Truncation-error order, grid-refinement convergence, conservation stability.
`perf-issue`	Profile and remove a bottleneck while asserting L_2 parity with the previous output.
`hillclimb`	Iterate against a quantitative target: one change, one measurement, keep or revert, with a decision log.
`bug-fix`	Reproduce on a minimal analytic case, find the root cause by evidence, failing test first.
`refactoring`	Pin behavior with a characterization test, then restructure into stateless physics modules.

Chapter 5

Safety Hooks and Tools

5.1 PreToolUse Safety Gate

`scripts/hook_pre_tool_guard.py` runs before `write_file`, `edit_file` and `run_command` calls and asks about proposed code that contains one of three hazards (as `force_ask`, so a cached "Always Allow" does not skip it):

1. **Explicit matrix inversion**: `numpy.linalg.inv`, `scipy.linalg.inv`, `torch.linalg.inv`, `torch.inverse`, and the JAX/CuPy/SciPy-sparse equivalents. Names are resolved through imports, so `import scipy.linalg as la; la.inv(A)` and `from numpy.linalg import inv; inv(A)` are caught, while an unrelated function called `inv` is not.
2. **Truncated π**: the literals `3.14159` and `3.1416`, outside tests.
3. **Mutable NumPy default argument**: `def f(x=np.zeros(3))`.

Comments, docstrings and strings are parsed away by the AST and never trigger the gate. For an `edit_file`, the imports of the file already on disk are used. Text that does not parse (a partial snippet) falls back to a conservative pattern match on comment-stripped text. `run_command` is checked best-effort: only when the command both writes to a scientific source file and contains an inversion call. The hook fails open on any internal error. Files named `hook_pre_tool_guard`, `audit_numerical_ast` or `inversion_detect` may name the patterns on purpose.

Where the inverse itself is the result, a line ending in the comment `# sciencestack: inverse-is-the-result` passes, and the auditor lists it as `ALLOWED_MATRIX_INV`. A negative `matrix_power` and `.I` of a `numpy.matrix` count as inversion; `pinv` does not. The parsed checks and the auditor apply to Python only: other languages get a regular expression in the gate and nothing in the auditor.

5.2 Norms, order on your own problem, and what a tolerance means

`compare -norm l1|l2|linf -norm-tol X [-blocks N]` judges a relative norm per field, for a new scheme against a reference solution; elementwise comparison stays the rule for ports. `convergence-user -exact FILE:FUNC -order P` measures the order of the user's solver against the user's exact or manufactured solution, whose file becomes a primary subject of the evidence; `mms_generator.py -output` writes that module and prints the command. `benchmarks/cfd/sod_calibration.` prints what a Sod tolerance means: a shock one cell out of place has a relative L1 error between 0.015 and 0.042, so `-tol 0.05` cannot see it. The withheld-verdict history is per solver file and function: a renamed or copied solver starts a new one, and records from before version 0.23 do not carry over.

5.3 Building both sides of a comparison

A manifest `compare` check can produce its reference and/or its test output by running programs instead of trusting committed files: `ref_from` and `test_from` give argument lists (never a shell, and a step that starts one, or holds a line break, is refused; `{work}` and `{repo}` are substituted; `LC_ALL=C`; a timeout per step), the output (`stdout` or a file in the work directory), and `pinned`, the SHA-256 of every source and input the numbers come from. A pinned file that changed fails the check before anything runs. For `ref_from` the committed reference must reproduce the rebuilt numbers within `reproduce_rtol` and `reproduce_atol`, stated separately because compilers and flags do not agree bit for bit, and the rebuilt reference is what is compared. `run-manifest -refresh-reference ID` rewrites the committed reference from its recipe for a person to review. Evidence is bound to the pinned files, and the ratchet treats a changed recipe as a changed reference. The steps are commands run with the caller's privileges: run only a manifest you trust, and do not run it in CI for pull requests from untrusted forks.

5.4 Verification hooks

In a git project that already uses sciencestack verification (it contains `reference-checks.json`, `test-ratchet-approvals.json` or a `.verification/` directory) more hooks act; elsewhere they do nothing. `scripts/hook_test_guard.py` (PreToolUse) compares a test file as it would be after an `edit_file`, `write_file` with the file now, using the same comparison as `test_ratchet.py`, and asks the person if a test would be weakened (the reason names the way forward: make the change yourself, record an exception in the approvals file, or set `SCIENCESTACK_HOOKS=off`; a missing or unknown `permission_mode` keeps the ask); it also asks before changes to the manifest that raise a tolerance, remove a check or add an `approved_relaxation`, to the approvals file, to golden files named in the manifest and to `.verification/`, and, best effort, before a shell command that writes to such a file. `scripts/hook_record_edit.py` (PostToolUse) records the project files this session changed; `scripts/hook_stop_gate.py` (Stop) then checks the evidence log and blocks once if an edited solver or compared output has no fresh passing check, a recorded test run is stale, or edited test files have no fresh ratchet run. It never blocks inside its own continuation (`stop_hook_active`) or while background tasks are pending, and raises each situation once.

Once a script is running, invalid input produces a prompt (or one block) showing the error. The recorder also runs after `run_command` and remembers the newest evidence-log line, and the stop gate blocks if a line it saw has gone. `SCIENCESTACK_HOOKS=off` in Antigravity's environment is the person's escape hatch for every hook. They cannot see edits made through other programs, and the stop gate cannot see a claim made without an edit; `test_ratchet.py` and CI do not depend on hooks.

Numerical Safety Alert: Limits

The gate cannot see an inversion built dynamically (`getattr`), or code written by a program that the agent launches. It is a safety net, not a proof of numerical soundness.

5.5 AST Auditor

`python3 scripts/audit_numerical_ast.py <file|dir> [-strict] [-json]` reports: `BAN_MATRIX_INV` (error); `CANCELLATION_LOG1P`, `CANCELLATION_EXPM1`, `CANCELLATION_ONE_MINUS_COS`, `PRECISION_FLOAT32`, `TRUNCATED_PI`, `MUTABLE_ARRAY_DEFAULT` (warnings); `MISSING_LATEX_PROVENANCE` and `ALLOWED_MATRIX_INV` (info). The exit code is 1 on errors, or on any finding with `-strict`.

5.6 MMS and Jacobian Generators

```
python3 scripts/mms_generator.py --exact "sin(pi*x)*exp(-t)" \
    --operator "diff(u, t) - alpha*diff(u, x, 2)" --params "alpha=0.1"
       --output mms.py
python3 scripts/derive_jacobian.py --vars "x,y" --eqs "x**2 + y,
   sin(x)*y"
```

For $u = e^{-t}\sin \pi x$ and $\mathcal{L}[u] = u_t - \alpha u_{xx}$ the generator derives the forcing

$$f(x,t) = (\alpha\pi^2 - 1)\, e^{-t}\sin \pi x,$$

and writes a module with `exact_solution` and `source_term`, whose arguments are the coordinates in a stated order (those the expressions use, or `-vars`). It prints the `convergence-user` command that measures a solver's order against the module. A parameter with no value is refused. The Jacobian generator emits `compute_jacobian(_state)` and the symbolic entries.

5.7 Reference Benchmarks

`scripts/check_against_reference.py` calls the user's solver and exits 0 only on a pass; `list` prints each contract, reference and tolerance rule.

Check	Reference	Tolerance
`kepler`	bisection on Kepler's equation, and its residual	default 10^{-12}
`lane-emden`	closed forms ($n = 0, 1$); high-accuracy integration ($n = 1.5, 3$)	required
`sod`	exact Riemann solution (Toro 2009)	required, relative L1
`robertson`	three implicit integrators; $y_1 + y_2 + y_3 = 1$	required
`beam-timoshenko`	$\delta = PL^3/(3EI) + PL/(\kappa GA)$	default 10^{-12}
`linear-solve-hilbert`	relative residual on H_n, $n = 3 \ldots 12$; forward error and exact κ_∞ reported, not judged	default 10^{-12}
`cavity-ghia`	Ghia, Ghia & Shin (1982), Table I, Re $= 100, 1000$	required, at least 10^{-3}
`convergence-ode`	four ODEs with exact solutions	claimed order
`convergence-bvp`	two 1-D Poisson problems	claimed order
`poisson-2d`	$u = \sin \pi x \sin \pi y$ on the unit square	claimed order
`convergence-user`	the user's exact or manufactured solution	claimed order
`compare`	the user's golden data	elementwise, or a norm

Table 5.1: Checks a solver can be run against.

The Ghia table was taken from two independently kept copies compared by program, not from the paper itself, and it is a 129×129 computation rather than an exact solution. In the tests a separate stream-function solver reproduces the Re $= 100$ column to 3.8×10^{-3} on 64 cells; Re $= 1000$ is not re-derived. On the Hilbert matrices at $n = 12$, the measured residual was 1.4×10^{-16} through `solve` and 3.5×10^{-3} through the inverse (NumPy, one platform).

The scripts under `benchmarks/` are the reference solvers behind these checks and worked examples:

Script	Reference	Source
`cfd/exact_riemann.py`	Sod star state $p_* = 0.30313$, $u_* = 0.92745$ ($\gamma = 1.4$)	Toro (2009), Table 4.1
`astronomy/lane_emden_solver.py`	$\xi_1 = \pi$ for $n = 1$; 3.65375 for $n = 1.5$; 6.89685 for $n = 3$	standard polytrope values
`astronomy/kepler_solver.py`	conserved E and $\mathbf{L}$	analytic invariants
`astronomy/symplectic_nbody.py`	bounded energy error, conserved $\mathbf{P}$, $\mathbf{L}$	symplectic structure
`astronomy/coordinate_transforms.py`	$R^T R = I$, $\det R = 1$	SO(3)
`kinetics/robertson_reference.py`	$y_1 + y_2 + y_3 = 1$	mass conservation
`cfd/sod_calibration.py`	relative L1 of a displaced shock and of a first-order scheme	Harten, Lax & van Leer (1983); Davis (1988)

Table 5.2: Reference solvers and what each is checked against.

Chapter 6

Testing

```
pytest tests/
```

The tests cover the safety hook (including the cases a substring match gets wrong), the AST auditor, the MMS and Jacobian generators (the Jacobian is compared with central finite differences), the reference benchmarks, the evidence log, the ratchets and the hooks, and the consistency of the documentation: every script flag written in a skill, agent or README exists, and no playbook points at a skill that does not ship. Tolerances are stated next to each expectation in the test sources. Reference values are re-derived by independent means (closed forms, sympy, mpmath, a different integrator), and every benchmark is tested with wrong solvers that must fail. `scripts/run_tests.py` reports the current count and records it, and `scripts/test_ratchet.py` refuses a change that weakens the suite.

Recommended Literature & References

1. Trefethen, L. N., & Bau III, D. (1997). *Numerical Linear Algebra*. SIAM.
2. Golub, G. H., & Van Loan, C. F. (2013). *Matrix Computations*, 4th ed. Johns Hopkins University Press.
3. Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
4. Hairer, E., Lubich, C., & Wanner, G. (2006). *Geometric Numerical Integration*, 2nd ed. Springer.
5. Roache, P. J. (2009). *Fundamentals of Verification and Validation*. Hermosa Publishers.
6. Salari, K., & Knupp, P. (2000). *Code Verification by the Method of Manufactured Solutions*. Sandia National Laboratories, SAND2000-1444.
7. Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics*, 3rd ed. Springer.
8. Hairer, E., & Wanner, G. (1996). *Solving Ordinary Differential Equations II*, 2nd ed. Springer.
9. Ghia, U., Ghia, K. N., & Shin, C. T. (1982). High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. *J. Comput. Phys.* 48, 387–411.
10. Timoshenko, S. P. (1921). On the correction for shear of the differential equation for transverse vibrations of prismatic bars. *Phil. Mag.* 41, 744–746.
11. Choi, M.-D. (1983). Tricks or treats with the Hilbert matrix. *Amer. Math. Monthly* 90, 301–312.
12. Harten, A., Lax, P. D., & van Leer, B. (1983). On upstream differencing and Godunov-type schemes for hyperbolic conservation laws. *SIAM Review* 25, 35–61.
13. Davis, S. F. (1988). Simplified second-order Godunov-type methods. *SIAM J. Sci. Stat. Comput.* 9, 445–473.